

LAB 3 — COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

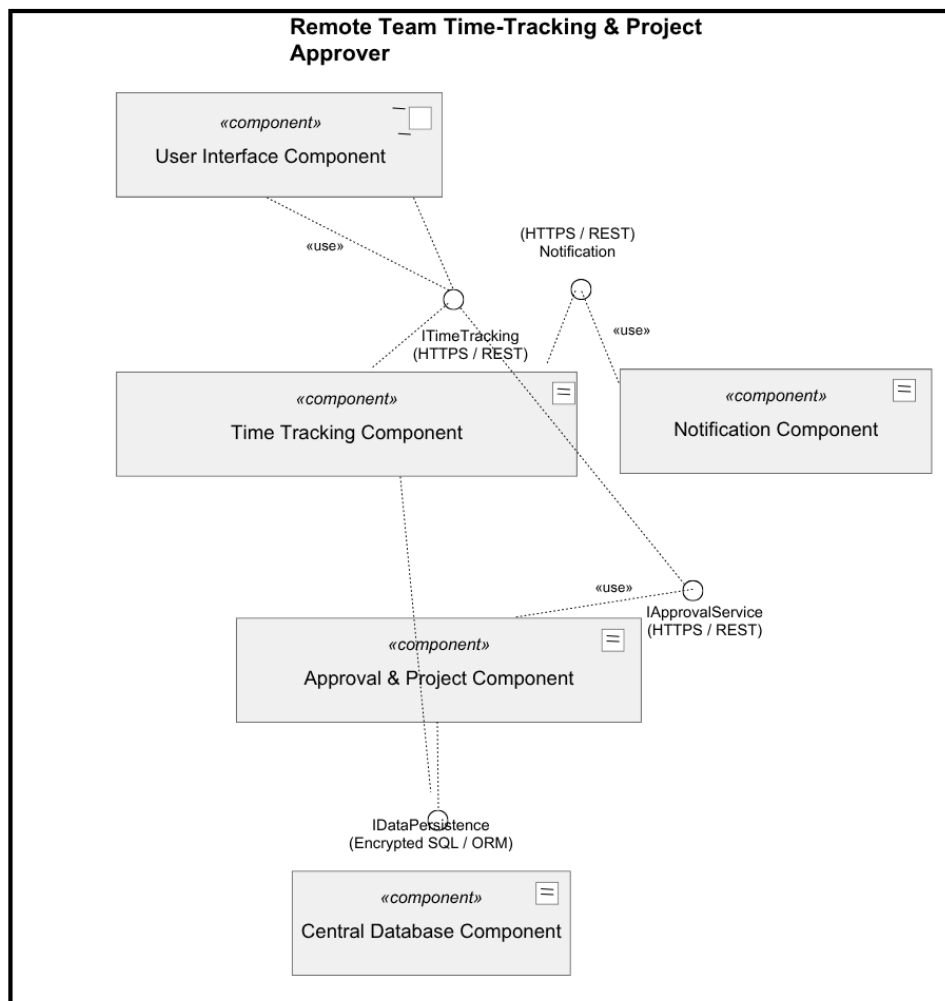
Sampada K • PES1UG24CS415 • Remote Team Time-Tracking & Project Approver

1. Architecture Comparison

Architecture	Advantages for this scenario	Limitations
Layered	Clear separation between presentation, business logic and data; straightforward maintenance.	Cross-layer calls can add overhead; individual functions are less independently scalable.
Microservices	Independent deployment/scaling and fault isolation for separate services.	Adds operational complexity, network calls and data-consistency concerns for a focused system.
Client-Server	Centralized timesheet/project data and access control; suitable for remote users connecting to a common server.	Central server can become a bottleneck or single point of failure without redundancy.

2. Component Diagram

The diagram below follows the requested component-diagram layout: system boundary, five components, four labelled interfaces, and «use» dependencies.



3. Components and Interfaces

Component	Responsibility
User Interface Component	Provides screens for logging time, submitting timesheets, viewing project information and checking approval status.
Time Tracking Component	Creates, validates, updates and retrieves time entries and timesheet totals.
Approval & Project Component	Routes submitted timesheets to project approvers and records approve/reject decisions and project-level status.
Notification Component	Sends submission, approval, rejection and correction notifications.
Central Database Component	Stores users, projects, time entries, timesheets, approvals and status history.

Interfaces shown: ITimeTracking (HTTPS/REST), IApprovalService (HTTPS/REST), INotification (HTTPS/REST), and IDataPersistence (Encrypted SQL/ORM).

4. Architectural Justification

Architecture Selection: We chose Client-Server architecture for the Remote Team Time-Tracking & Project Approver System.

Reason 1: Remote team members and project approvers need a common source of truth for timesheets, approvals and project data. Centralizing these records on the server reduces inconsistent copies of approval status.

Reason 2: The system has role-specific actions, so centralized authentication, authorization and validation can be applied before changes reach shared project data.

Security Advantage: The server can enforce role-based access to time records and approval operations while keeping the database behind the application layer.

Performance Benefit: The client communicates with focused server components using lightweight request/response payloads. Indexed database queries and efficient data retrieval can keep routine time-entry and submission operations responsive.\